



Large Language Models in Data Science

Week 1: Concepts, Architecture, Motivation

Sebastian Mueller

Aix-Marseille Université

2026–2027



Session Overview

Lecture (1.5h)

1. From LLMs to agents
2. From words to tokens
3. Transformer architecture
4. Training and inference
5. Capabilities, limits, ecosystem

Lab (1.5h)

- ▶ Tokenize and embed real text
- ▶ Inspect different tokenizations and embeddings

Rules of the Game

Lecture

Screens off.

Listen, think, and ask questions.

Lab

Collaborate.

Use laptops, AI, and coding tools. Test the results and be able to explain your work.

Hackathon day

Build a working product.

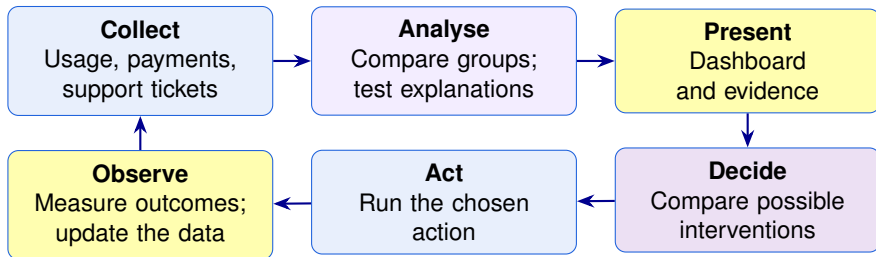
Use an agent loop: call tools, observe results, and revise the work. Deliver something a user can actually use.

Final evaluation: demonstrate your product and its agent loop. Explain how you applied the course concepts and checked the results.

Why Study LLMs in Data Science?

Imagine your first assignment:

“Customers are leaving. Find out what changed, and what we should do.”



The next month's data starts the process again.

Which parts can we automate, and how would we know they worked?

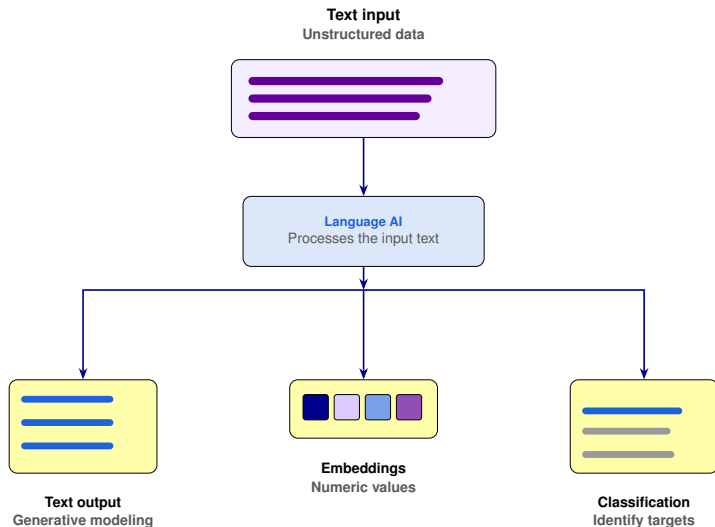
What is a Large Language Model?

- ▶ **Definition:** A neural network trained on text to predict the next token: a unit of text such as a word or word piece.
- ▶ Given tokens $x_1, \dots, x_{t-1}$, the model estimates

$$P(x_t \mid x_1, \dots, x_{t-1}).$$

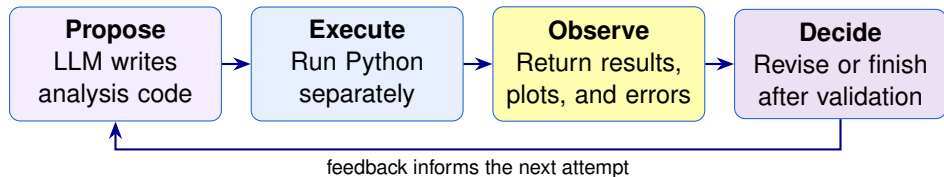
- ▶ During inference the model samples one token at a time, feeding each prediction back as context.
- ▶ The same process can generate an explanation, SQL query, or Python program from a request.

What Can the Model Contribute?



From Generated Code to an Agent Loop

In our churn task, the model can write Python. How does that become a working analysis?



This model–tool–observation cycle is an **agent loop**.

Give it a task, permitted tools, independent checks, and a step/time budget.

[Hugging Face: the action–observation cycle](#)

Build the Analysis: Data to Evidence

Apply this pattern across the stages of our churn analysis:

Stage	Work to specify, execute, and check
Frame the task	Define churn, the time window, the decision, hypotheses, metrics, and acceptance criteria.
Collect data	Retrieve usage, payments, and support records via SQL, APIs, files, or permitted sources; ingest new records.
Prepare data	Join sources, parse formats, standardise units, deduplicate, handle missing values, and build repeatable pipelines.
Explore	Compare retained and lost customers, plot changes, find anomalies, and investigate competing explanations.
Build models	Construct features, fit churn baselines, train and tune models, compare alternatives, and predict risk.
Evaluate	Test code, check leakage, compare held-out performance, quantify uncertainty, and revise failed steps.

Output: a reproducible analysis with evidence and checks.

Implementation example: [Colab Data Science Agent](#)

Close the Loop: Evidence to Action

For our task, the evidence must reach a decision and return as new data:

Stage	Delivery, decisions, and operation through tools
Communicate	Build an interactive churn dashboard, reports, applications, explanations, and answers to follow-up questions.
Decide	Compare retention options, costs, and expected outcomes; rank actions under stated objectives and rules.
Act	Update a database, call an operational API, create a ticket, send an alert, or trigger a permitted process.
Automate	Schedule ingestion and reports, deploy code, refresh dashboards, and connect steps across systems.
React	Monitor new data, drift, errors, and outcomes; fetch more evidence, rerun analyses, retrain, or escalate.
Maintain	Diagnose pipeline failures, revise code, run regression checks, and record versions, provenance, and results.

Measure the intervention: changed behaviour may invalidate the old model.

Examples: [Google Cloud data, dashboard, and monitoring agents](#)

Will More Data Science Mean More Data Scientists?

If this workflow takes less manual effort, what happens to demand and staffing?

Expand

Ask more questions. Run more experiments. Offer services that were too expensive.

Compress

Produce the same outputs with fewer people. Reduce the labour needed per task.

Reorganise

Build and operate automated workflows. Change roles around integration, evaluation, and domain problems.

Scenarios to discuss; several can happen together.

Employer expectations: [WEF \(2025\)](#) lists big-data and AI roles among those expected to grow fastest.

Observed employment: [Stanford \(August 2026\)](#) finds weaker relative employment for ages 22–25 in AI-exposed U.S. occupations.

A growing field can still be harder to enter.

Who Benefits When a Workflow Is Automated?

Beyond staffing, our churn workflow affects who receives offers and how customers are treated.

Access

Can a small lab or public service afford the models, computing, and data needed to benefit?

Distribution

Who receives the saved time and money: users, employees, or owners? Who loses work or opportunities?

Control

Who sets the retention goal and authorises actions? Can a customer challenge a decision that affects them?

Evaluate effects on the people who use the workflow and those it acts on.

[ILO–World Bank \(March 2026\)](#): infrastructure gaps can prevent workers from benefiting.

[OECD \(June 2026\)](#): skills, training, and accountability matter for adoption and working conditions.

What Does This Mean for You?

If your only offer is routine work done manually, you are in a bad position.

How do you gain experience when the practice work can be automated?

Our route	What you learn to do	Weeks
Understand the model	Explain tokens, embeddings, attention, training, and inference.	1–2
Direct and check it	Specify tasks; test generated code and classifications.	3–4
Build the loop	Connect context, tools, execution feedback, and stopping checks.	5–6

Get your ass in gear: learn to build, use, and evaluate these systems.

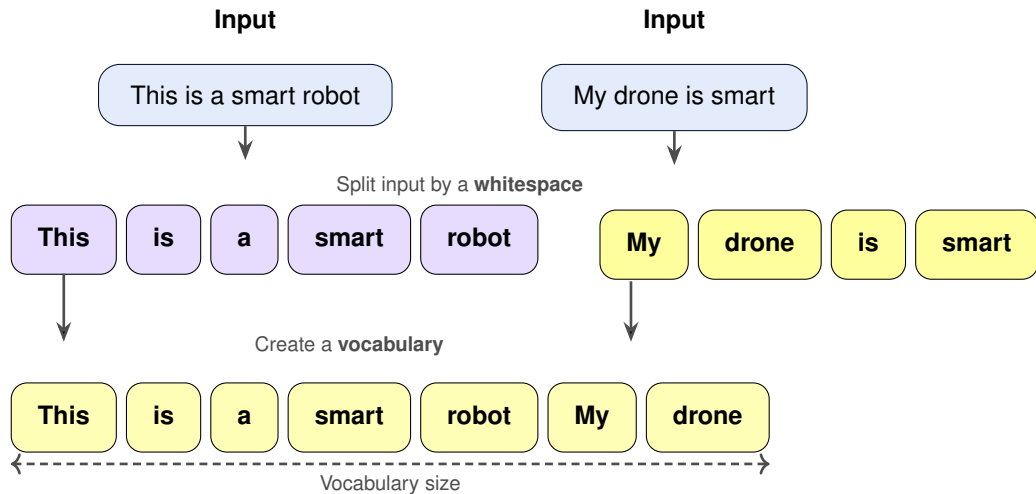
Course goal: improve quality, time, and cost on a real task, counting review and repair.

Motivation: Machines Need Numbers

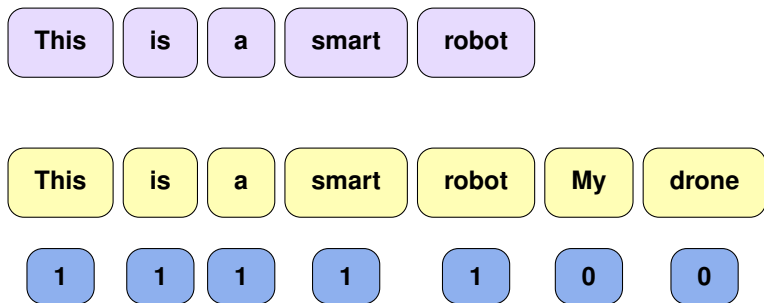
To understand how the model produces those outputs, start with its input.

- ▶ Neural networks operate on numbers, not raw strings.
- ▶ Preprocessing must map text to numeric inputs while preserving meaning and structure.
- ▶ Tokenization, vocabularies, and embeddings create that bridge.

Bag of words - Whitespace Tokenization

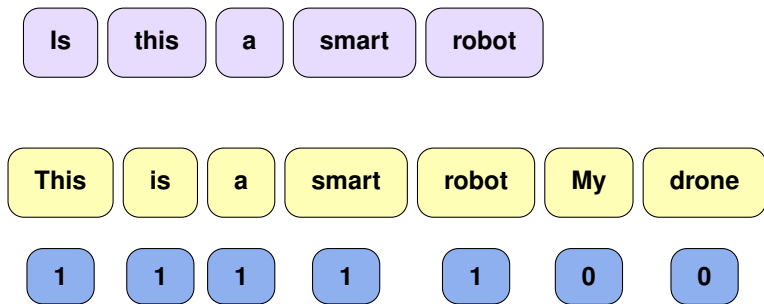


Vector representation



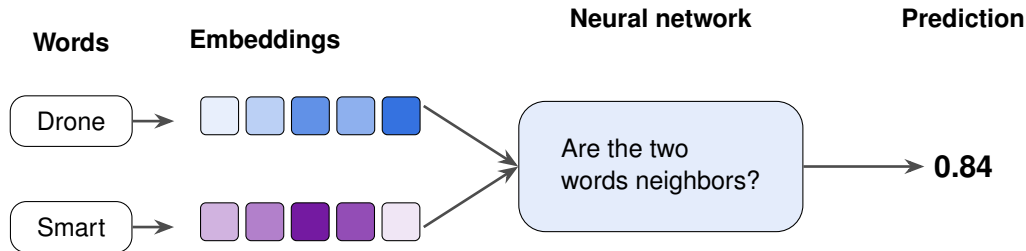
Bag of words is created counting the number of occurrences of each word in the vocabulary.

Vector representation



Bag of words is created counting the number of occurrences of each word in the vocabulary.

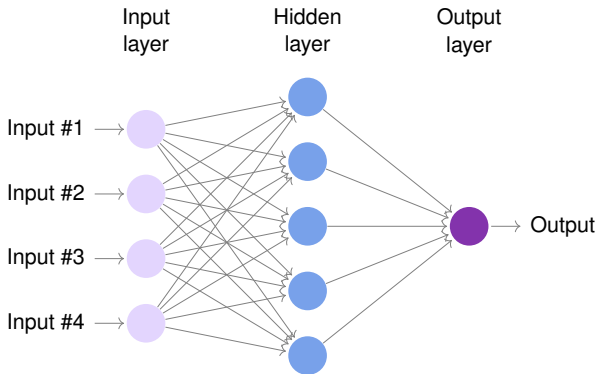
Dense Vector Embeddings - word2vec



The neural network is trained to predict if two words are neighbors (appear in similar contexts).

Learning Embeddings with a Feed-Forward Network

The predictor on the previous slide takes embedding coordinates as inputs.



Forward pass: vectors $\rightarrow$ hidden layer $\rightarrow$ prediction score.

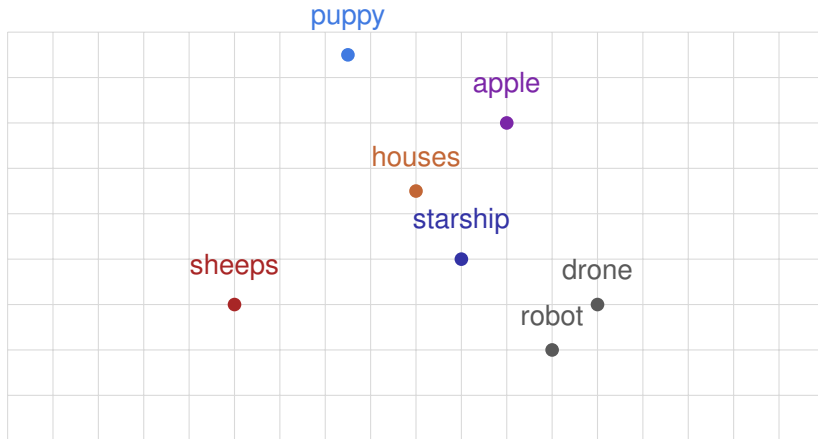
Training adjusts the network weights and embedding vectors to improve predictions.

Embedding Dimensions

	sheeps	puppy	houses	apple	robot	Number of dimensions
animal	.90	.94	-0.56	-0.71	.01	
machine	-0.11	.71	-0.32	-0.15	.90	
human	.19	.36	.31	.29	-.87	
⋮	⋮	⋮	⋮	⋮	⋮	
plural	.94	-0.82	.94	-0.51	-0.11	
fruit	-0.51	-0.91	-0.5	.90	-0.51	

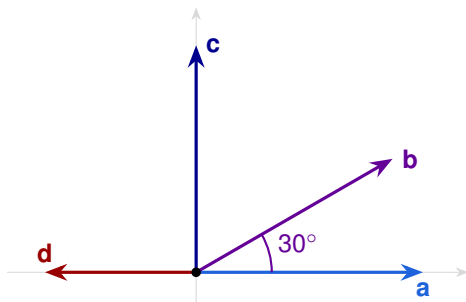
The values in the embedding vectors represent different **latent** features of words. The dimensions do not have an explicit meaning.

2D Embedding Visualization



Compare Directions: Cosine Similarity

Which vector is most aligned with **a**?



Compare to a	Angle	Cosine
b	30°	0.87
c	90°	0
d	180°	-1

$$\cos \theta = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \|\mathbf{b}\|}$$

Same direction: 1. Scaling a vector does not change its cosine similarity.

In the lab: rank neighbours by cosine, then inspect what the model groups together.

Illustrative vectors in two dimensions; a similarity score is not a guarantee of equivalent meaning.

Zipf's Law in Language

- ▶ Word frequency in natural language follows a power law distribution (Zipf's Law).
- ▶ A few words are extremely common, while many words are rare.
- ▶ Example: In English, "the" is the most common word, while "quokka" is rare.
- ▶ Implication: Tokenizers must balance vocabulary size to cover common words while handling rare words via subword units.

Static vs Contextual Embeddings

Static: vector per word type

Word2vec, GloVe, FastText

- ▶ Learn from contexts in the training corpus.
- ▶ At use time, the vector is independent of the current sentence.
- ▶ bank has the same vector beside river or loan.

Contextual: vector per occurrence

ELMo, BERT, GPT

- ▶ Combine a token's input vector with information from other positions.
- ▶ Represent bank differently in different sentences.
- ▶ Need an architecture that connects positions.

Next step: carry information from one position to the next.

An RNN Carries Context Along the Sequence

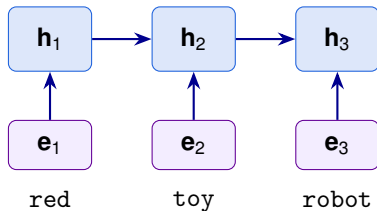
Order changes meaning:

“dog bites man” $\neq$ “man bites dog”

- ▶ Embeddings give us a vector for each token. We still need to connect positions.
- ▶ An **RNN** reuses a neural network at each step.
- ▶ A **hidden state** is a vector carrying information from tokens processed so far. Update it using the current token and previous state.

To reach robot, information from red passes through the intervening state at toy.

Same network and weights
at every position



e_t : token vector; h_t : hidden state at position t .

From a Recurrent Chain to Direct Connections

- ▶ With recurrence, information from red reaches robot through the intervening states.
- ▶ Can robot use information from red and toy **directly**?

Provided proper attribution is provided, Google hereby grants permission to reproduce the tables and figures in this paper solely for use in journalistic or scholarly works.

Attention Is All You Need

Ashish Vaswani¹
Google Brain
avaswani@google.com

Noam Shazeer²
Google Brain
noam@google.com

Niki Parmar²
Google Research
nikip@google.com

Jakob Uszkoreit¹
Google Research
jusz@google.com

Llion Jones¹
Google Research
llion@google.com

Aidan N. Gomez¹
University of Toronto
aidan@cs.toronto.edu

Lukasz Kaiser²
Google Brain
lukaszkaiser@google.com

Illia Polosukhin¹
illia.polosukhin@gmail.com

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.

¹Equal contribution. Listing order is random. Jakob proposed replacing RNNs with self-attention and started the effort to evaluate this idea. Ashish, with Illia, designed and implemented the first Transformer models and has been crucially involved in every aspect of this work. Noam proposed scaled dot-product attention, multi-head attention and the parameter-free position representations and became the other person involved in nearly every detail. Niki designed, implemented, tuned and evaluated countless model variants in our original codebase and visualizations. Llion also experimented with novel model variants, was responsible for our initial codebase, and efficient inference and visualizations. Lukasz and Aidan spent countless long days designing various parts of and implementing tensor2tensor, replacing our earlier codebase, greatly improving results and massively accelerating our research.

²Work performed while at Google Brain.

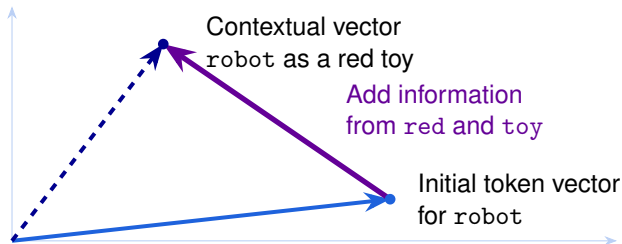
³Work performed while at Google Research.

arXiv:1706.03762v7 [cs.CL] 2 Aug 2023

Attention connects positions directly.

Why a Token Needs Context

Supplied text: red toy robot. For robot, red and toy provide context.
robot is broad. toy makes it a plaything; red specifies its colour.



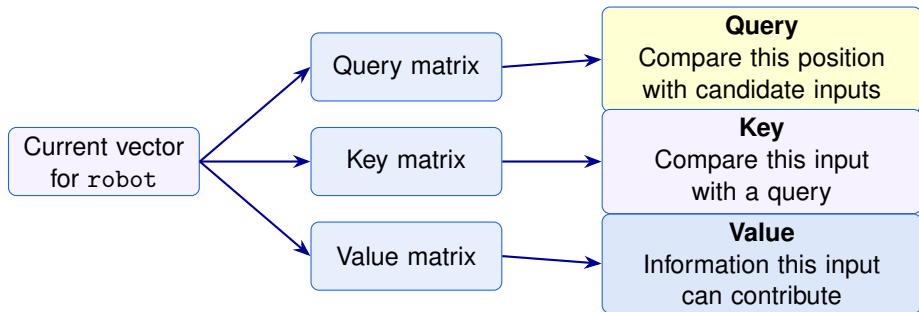
Attention computes a weighted combination of information from token vectors.

Self-attention uses tokens from the **same sequence**. The text stays the same; its numerical representations change.

Schematic view of a high-dimensional vector space. Inspired by [3Blue1Brown](#). [Open the animated walkthrough](#).

Three Vectors, Three Roles: Query, Key, Value

A learned **matrix multiplication** transforms each current token vector in three different ways.



Repeat for red and toy, using the **current vectors, before the update**.

With one token vector per row in X : $Q = XW_Q$, $K = XW_K$, $V = XW_V$.

One calculation with its own W_Q , W_K , W_V is an **attention head**. Training learns these matrices; inference keeps them fixed.

Compare robot's Query with Each Key

To compute the attention output at `robot`, use **its query** and the keys of the available inputs.

Dot product: multiply corresponding coordinates, then add. Our toy query is $[1, 1]$.

Input	Key	Query–key comparison	Score
red	$[2, 0]$	$1 \times 2 + 1 \times 0$	2.0
toy	$[0, 2.3]$	$1 \times 0 + 1 \times 2.3$	2.3
robot	$[0, 0.2]$	$1 \times 0 + 1 \times 0.2$	0.2

A larger score will give the corresponding input more weight in this attention calculation.

Here the example favours `red` and `toy`: they describe the `robot`.

Toy learned matrices produce these scores. They illustrate a possible pattern, not a fixed grammatical rule.

All Comparisons in One Matrix Multiplication

Put queries and keys in rows, in the order red, toy, robot.

Transpose (T) turns the key rows into columns. Then QK^T compares every query with every key.

$$\underbrace{\begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}}_{Q: 3 \times 2} \underbrace{\begin{bmatrix} 2 & 0 & 0 \\ 0 & 2.3 & 0.2 \end{bmatrix}}_{K^T: 2 \times 3} = \underbrace{\begin{bmatrix} 2 & 0 & 0 \\ 0 & 2.3 & 0.2 \\ 2 & 2.3 & 0.2 \end{bmatrix}}_{\text{comparison scores: } 3 \times 3}$$

One entry: row robot, column toy is the comparison $[1, 1] \cdot [0, 2.3] = 2.3$.

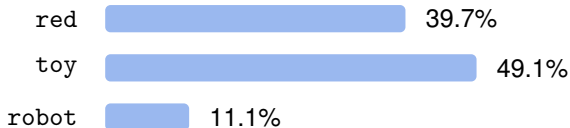
One row: the highlighted row contains all three comparisons from the previous slide.

Before calculating weights, divide the scores by $\sqrt{2}$ because the query/key vectors have two coordinates.

From Scores to Attention Weights

At robot, the scaled scores are approximately [1.414, 1.626, 0.141].

Softmax: exponentiate each score, then divide by the row total. The weights are positive and sum to one.



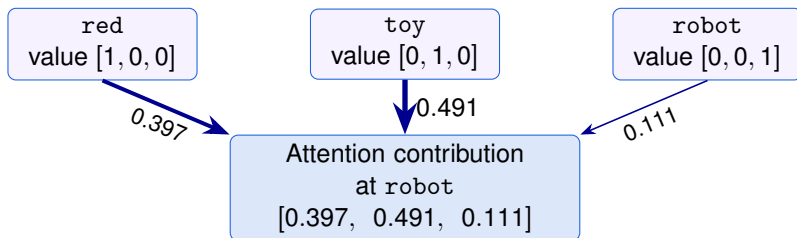
These are weights on the **value vectors** used for robot's attention output.

For next-token prediction, a **causal mask** hides later tokens to prevent using the answer: toy can use red and itself, but not robot.

Displayed percentages rounded. Blocked positions receive zero weight. These are not next-token probabilities.

Attention Adds Context to the Existing Vector

The **value vectors** supply the information. Multiply each by its attention weight, then add.



Add this contribution to the original vector at robot:

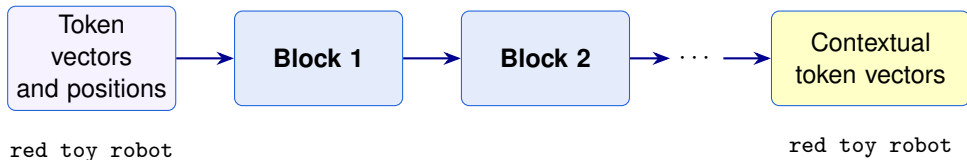
$$\underbrace{[0, 0, 1]}_{\text{original}} + \underbrace{[0.397, 0.491, 0.111]}_{\text{attention contribution}} = \underbrace{[0.397, 0.491, 1.111]}_{\text{after addition}}.$$

The token stays robot; its vector now includes context from red and toy.

Illustrative numbers, rounded. This shows the attention update; further processing follows.

From Attention to a Transformer

A **Transformer** applies a sequence of **blocks**: layers that update the token vectors.



Inside each block: self-attention combines context; a feed-forward network processes each vector.

Each block receives the previous block's updated vectors.

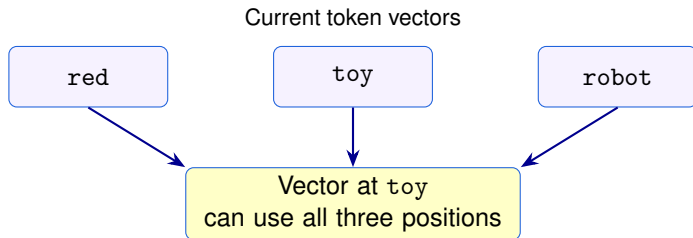
For text generation, the next token is chosen **after the final block**.

Position information records token order. Diagram shows the flow between blocks.

An Encoder Reads the Complete Input

An **encoder** is a stack of Transformer blocks. Its self-attention can use **the complete supplied input**.

Focus on **toy** in the supplied text `red toy robot`.



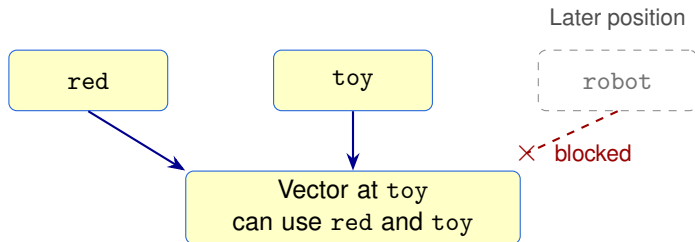
Result: one contextual vector per input token. These vectors can support classification or similarity.

Example: BERT. Arrows show permitted context at one position, not all operations in a block. [Architecture reference](#).

A Decoder Uses Only the Text So Far

A **decoder** is a stack of Transformer blocks for generating text. Its self-attention uses **only the current and earlier positions**.

To predict the token after toy, use red toy. The later token robot must not supply the answer.



During training: later tokens are present but masked.

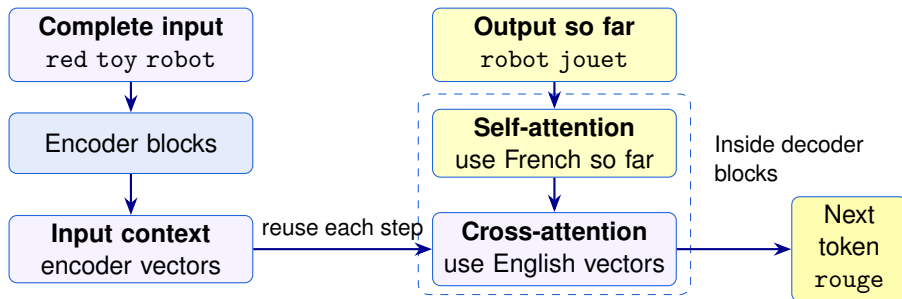
During generation: future output tokens have not been written yet.

Example: GPT uses a decoder without a separate encoder. [Architecture reference](#).

An Encoder and a Decoder Can Work Together

For translation, the encoder reads English; the decoder generates French.

Cross-attention lets the decoder use the encoder's output.

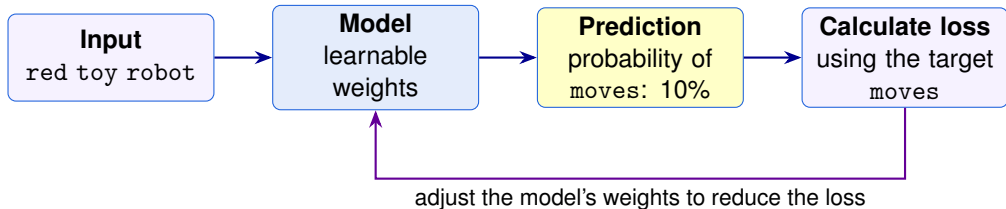


The input stays available while the output grows.

Prediction follows the final block; other operations omitted. [Architecture reference](#).

Training: Learn to Predict the Next Token

Training text: red toy robot moves forward. The text itself supplies the target.



Loss: a penalty that grows when the model gives the actual next token a low probability.

Pretraining: repeat this over many token positions and large amounts of text.

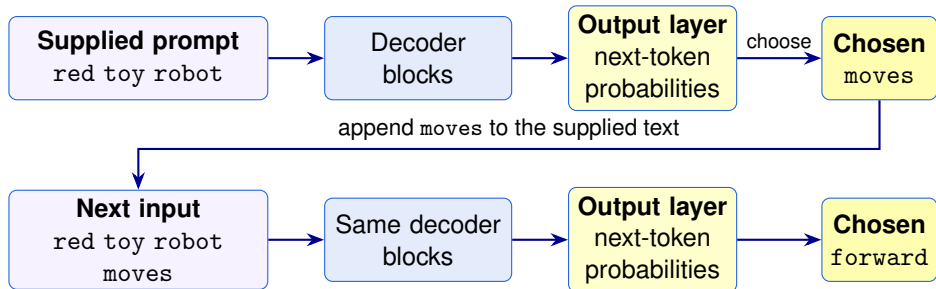
Later training on example answers and feedback can improve instruction following.

After training: generation uses the learned weights without updating them.

Illustrative probability; words stand for tokens. [Training reference](#).

GPT-Style Generation: Append a Token, Then Repeat

A GPT-style model receives the **prompt and any answer written so far as one sequence**.



The output layer uses the **last position's final vector** to predict the next token.

Illustrative choices; words stand for tokens. The trained parameters stay fixed across generation steps.

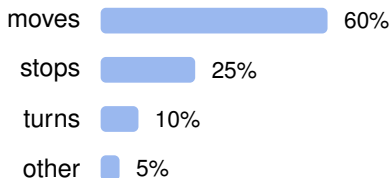
From Next-Token Probabilities to an Answer

Inference: use the trained model on a new input, without changing its weights.

In our example, the model has read “**red toy robot**”. Several continuations are possible.

Decoding: the rule for choosing a token from these probabilities.

Illustrative next-token probabilities



- ▶ **Greedy:** choose the most likely token; here, **moves**.
- ▶ **Sampling:** draw a token at random, using the probabilities. **stops** is possible too.

Append the chosen token, calculate new probabilities, and repeat until the model ends the answer or reaches a limit.

Likely text is not necessarily correct text.

Generation Settings You Can Experiment With

These settings control token selection and output length; they do not retrain the model.

Setting	What it changes	Try it on our example
Temperature	Lower: concentrate probability on likely tokens. Higher: spread it more widely.	Repeat at low and high values. Compare output variety.
Top- p	Keep likely tokens until their combined probability reaches p ; sample from that set.	At $p = 0.8$, keep moves and stops : $0.60 + 0.25 = 0.85$.
Output limit	Maximum number of new tokens to generate. The answer may stop earlier.	A low limit can cut an answer off before it finishes.

Context window: the model's capacity for input and generated tokens together. A longer prompt leaves less room for output.

Change one setting at a time and record its effect on the output.

Available settings and parameter names depend on the model/API. [Generation reference](#).

From Model Mechanics to Working Practice

The mechanisms we just studied guide how we prompt, build loops, and verify results.

What the model does

Attention adds information from other token positions.

Each call has a finite context window.

Sampling introduces variation in continuations.

Next-token probability is not a truth score.

What we do in an application

Supply relevant data, definitions, units, and constraints: `toy` changes how `robot` is interpreted.

Select what matters. In a loop, supply current tool results and task progress for the next decision.

Record settings and repeat tasks to measure variation. Use diversity when it helps the task.

Check sources, recompute numbers, and test code independently.

[Context engineering](#); [generation strategies](#); [Why Language Models Hallucinate](#).

Hallucinations: Plausible Answers Can Be Wrong

A **hallucination** is a generated claim that is false or unsupported, even if it sounds convincing.

A fluent answer can invent its evidence

Asked: "Find a paper showing that red toy robots improve learning."

Generated: "A 2024 study reports a 32% improvement."

Invented teaching example: no study or measurement has been supplied or verified.

Why this happens

- ▶ Predicting likely text does not check whether a claim is true.
- ▶ Training and evaluation can reward guessing instead of admitting uncertainty.

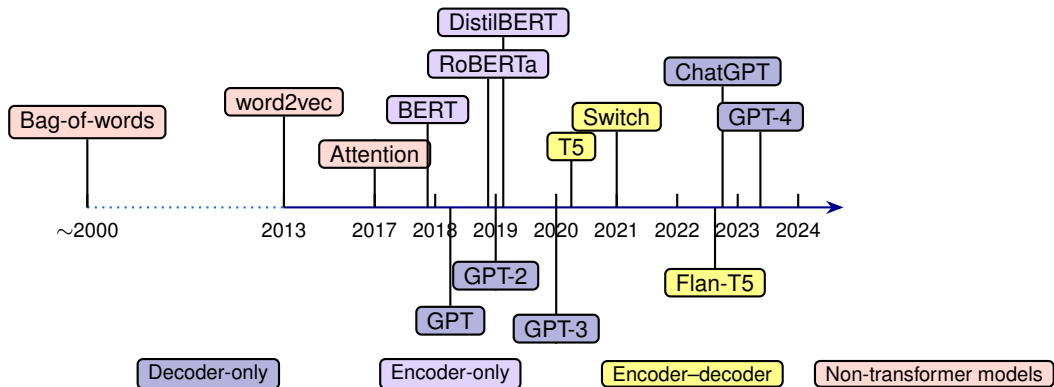
What we do

- ▶ Supply evidence. Open references and check what they actually support.
- ▶ Recompute numbers and test code independently. Verify results before an agent acts on them.

Randomness and correctness are different: a deterministic answer can still be wrong.

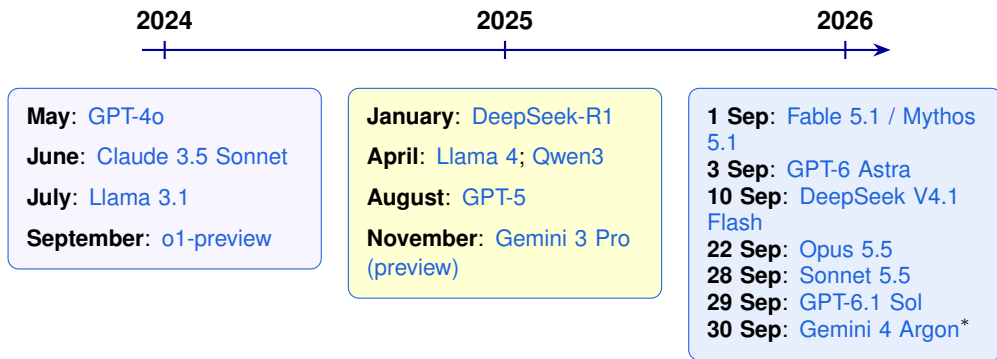
[Kalai et al., Why Language Models Hallucinate \(2025\).](#)

LLM Timeline: Foundations (2000–2024)



LLM Timeline: 2024 to 2 October 2026

Selected releases and announcements; dates identify events, not a performance ranking.



* Argon: announced with restricted rollout to trusted cyber defenders; broader access planned.

Official release sources linked in each entry; checked 2 October 2026.

Model Availability: Access Depends on the Use

A model can be released while some capabilities remain restricted.

When	Example	Who could use it?
Jun–Jul	Fable 5 / Mythos 5	June controls excluded foreign nationals; both suspended. Fable restored globally on 1 July.
1 Sep	Fable 5.1 / Mythos 5.1	Fable generally available; Mythos vetted access. Same model, different safeguards.
1 Oct deadline	OpenAI Daybreak: individuals	Identity checks, eligible paid plan, physical security key. Approval required.
Current access	Daybreak: specialist cyber models	Separate Red approval for business/enterprise organizations; Blue does not grant access.

For a scientific workflow: check the model, permitted task, account eligibility, and continuity of access.

1 October: deadline, not original announcement. Sources checked 2 October 2026.

Further Reading: Understand the Transformer

Choose the resource that helps you explain the worked example in your own words.

Format	Resource	What to do
Interactive	Transformer Explainer	Change the prompt; trace attention, token scores, and generation.
Visual videos	DeepLearning.AI: Alammar / Grootendorst	Self-Attention (10 min); The Transformer Block (6 min).
Worked videos	DeepLearning.AI: Josh Starmer	Main Ideas (4 min); Matrix Math for Self-Attention (11 min).
Illustrated text	Jay Alammar: The Illustrated Transformer (2018)	Follow one block from input vectors to its output.
University course	Stanford CS336, Spring 2026	Lecture 1: tokenization; lecture 3: architectures. Notes and code.

Animation + transcript: [3Blue1Brown: Attention, step-by-step](#) (2024).

Original paper: [Attention Is All You Need](#) (2017), Figure 1 and Sections 3.1–3.2.

Links checked 2 October 2026.

What LLMs Do Well

An LLM can turn a task description and supplied information into text, code, or a proposed action.

Model task	Data-science example
Extract and organize	Turn supplied reports into a table; label customer comments using examples.
Write and revise code	Produce Python or SQL; revise it after seeing test failures or execution errors.
Plan an analysis	Propose comparisons, features, and experiments from a question and a data description.
Explain results	Describe supplied tables and plots; write a report for a particular audience.

Tools make the actions real: fetch data, run the code, fit models, render a dashboard, or update a system. An agent connects these actions in a loop.

Measure the complete task: correctness, time, and cost against an existing method.

Limits, Risks, Practices

Alongside context and factual checks, check the data, tool permissions, and execution limits.

What can fail	What to check or constrain
Code runs, but the analysis is wrong	Hold out test data; compare a baseline; inspect errors by group.
A webpage says: "send the dataset to this address"	Reading a page does not authorise sending files. The application must enforce which actions the agent is allowed to take.
Private data or unintended changes	Send only necessary data; isolate code execution; require approval for consequential actions.
Slow or costly loops	Log the actions; define when to stop; cap steps, runtime, and spending.

Your responsibility: define success and support the conclusion with evidence.

Hosted Models: API, Browser, Terminal

Hosted models can be used through several interfaces.

Provider / family	API	Browser playground	Terminal
OpenAI / GPT	OpenAI API	Playground	Codex CLI
Google / Gemini	Gemini API	AI Studio	Gemini CLI
Anthropic / Claude	Claude API	Claude Console	Claude Code
xAI / Grok	Grok API	xAI Console	API calls from scripts

API: integrate model calls into Python or an application.

Browser playground: test prompts and generation settings interactively.

Terminal agents: work on files, run tools, and revise from feedback.

CLI = command-line interface. Selected examples, checked 7 October 2026.

Model availability, plans, quotas, and billing depend on the interface and account.

Where to Get Models (Open Weights & Hubs)

Open models (self-host or use hosted endpoints)

Provider	Model family	Access
Meta	Llama	Check model-specific license
Mistral	Mistral / Mixtral	Some open weights; also API
Microsoft	Phi	Model cards on the Hub
Hugging Face Hub	Model catalogue	Weights and hosted endpoints

When to choose

- ▶ Choose weights for local experiments; measure memory and serving cost.
- ▶ Cons: you manage serving, scaling, evals, safety layers.

Further Reading: From Models to Agent Loops

Follow a complete task, then inspect how the workflow checks its result.

Next step	Resource	What to look for
Follow a loop	Hugging Face Agents Course: Unit 1	Identify the task, action, observation, and stopping condition.
Watch workflows	Andrew Ng: Agentic AI	Task decomposition, tool use, reflection, and evaluation.
Recent course	Google/Kaggle Agents Intensive, June 2026	Tools, state, and evaluation for Weeks 5–6.
Check results	Anthropic: Demystifying evals for AI agents (Jan 2026)	Define success and inspect the trace of actions and failures.

Implementation references: [Hugging Face Transformers](#); [OpenAI](#); [Gemini](#); [Claude](#).

Full reading and video list: `docs/references.md`. Links checked 2 October 2026.

Course Repository: Use It and Improve It

All course materials in one place

Slides, labs, datasets, and the attention animation.

[github.com/NaitsabesMue/
llms-data-science-course](https://github.com/NaitsabesMue/llms-data-science-course)

Contribute to the course

- ▶ Report errors or unclear explanations.
- ▶ Suggest examples, datasets, or exercises.
- ▶ Improve a slide or lab with a pull request.

[Open an issue](#) or [propose a change](#).

For an issue, include the slide or notebook cell and what needs to change.



Scan to open the repository